

Оглавление

- 1 *Кибизов К.О., Сальников А.Н.* Технические аспекты выбора пути передачи данных через физическую среду в кластере с графическими процессорами 4

Кибизов К.О., Сальников А.Н.

ТЕХНИЧЕСКИЕ АСПЕКТЫ ВЫБОРА ПУТИ ПЕРЕДАЧИ ДАННЫХ ЧЕРЕЗ ФИЗИЧЕСКУЮ СРЕДУ В КЛАСТЕРЕ С ГРАФИЧЕСКИМИ ПРОЦЕССОРАМИ

Введение

При использовании нескольких GPU производительность зависит не только от скорости вычислений, но и от скорости обмена данными в коммуникационной среде: медленный обмен может стать узким местом.

При выполнении программы данные между GPU могут передаваться различными путями, в том числе с промежуточным копированием через оперативную память или без него. Их выбор определяется, в частности, расположением устройств, настройками транспортного слоя и привязкой ресурсов к процессам программы. От этого, в свою очередь, зависит задержка передачи, то есть время, необходимое для доставки данных от одного GPU к другому.

Для разных пар GPU задержка может заметно различаться. Неоднородность задержек между парами узлов кластера показана в [1], а в [2] предложен метод их измерения для GPU, расположенных на разных узлах. Настоящая работа продолжает этот подход и расширяет набор тестов проекта Clustbench в ветке `gpu_tests` [3]. Ранее на каждом узле запускался один MPI-процесс, обслуживавший все его GPU: обмен внутри узла измерялся средствами CUDA без участия MPI, а между узлами всегда шёл через оперативную память. Теперь каждому GPU соответствует отдельный MPI-процесс, поэтому любой обмен, как внутри узла, так и между узлами, измеряется через MPI, а при необходимости передачу можно принудительно вести через оперативную память.

Цель работы — исследовать задержки передачи данных между GPU в зависимости от размера сообщения, способа соединения GPU и среды передачи. Отдельное внимание уделено настройкам Open MPI, транспортного слоя UCX и планировщика Slurm, от которых зависит путь передачи.

1. Модель коммуникационной среды и постановка задачи

Коммуникационная среда представляется в виде графа. Его вершины соответствуют аппаратным компонентам: процессорам, GPU, сетевым адаптерам и коммутаторам. Рёбра соответствуют физическим соединениям: PCIe, NVLink, Ethernet, InfiniBand. При тестировании GPU обмениваются сообщениями — синтетически сформированными блоками данных заданного размера. При передаче сообщение проходит по графу через последовательность вершин и рёбер, то есть по некоторому пути. Множество возможных путей зависит от топологии и программного окружения, а конкретный путь выбирает транспортный слой. Вообще говоря, сообщение может быть разбито на фрагменты, которые передаются по разным путям.

Основные пути передачи данных между GPU показаны на рис. 1. Без промежуточного копирования данные передаются внутри узла средствами GPUDirect Peer-to-Peer [4] (путь а), между узлами — GPUDirect RDMA [5] (путь с). В остальных случаях данные копируются через оперативную память (пути b и d).

Предметом изучения служит сквозная задержка передачи сообщения между упорядоченной парой различных устройств, то есть время от начала передачи на устройстве-отправителе до завершения приёма на устройстве-получателе.

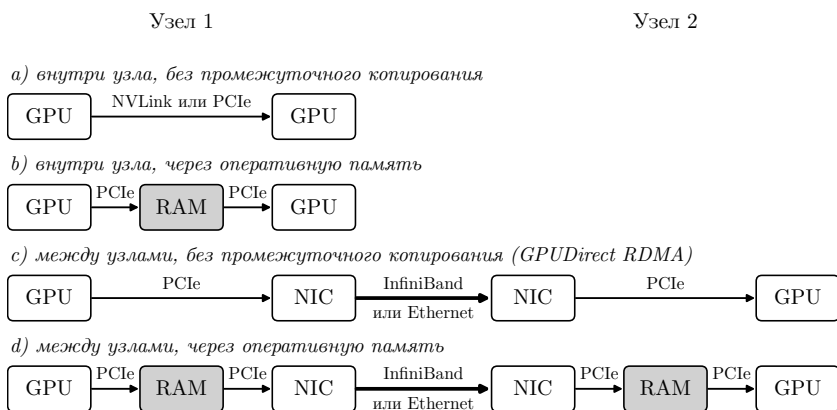


Рис. 1: Пути передачи данных между GPU (RAM — оперативная память, NIC — сетевой адаптер)

2. Способ измерения

Задержка измеряется на стороне получателя таймером `MPI_Wtime()`. Для упорядоченной пары GPU (i, j) в k -м замере получатель фиксирует момент t_k^{rdy} готовности принять данные и момент t_k^{end} завершения приёма, а задержка оценивается как

$$t_{ij,k} = t_k^{\text{end}} - t_k^{\text{rdy}}.$$

Завершение приёма имеет однозначный смысл: к этому моменту данные находятся в буфере получателя. Начальная отметка может предшествовать началу передачи, поэтому $t_{ij,k}$ является оценкой сверху для сквозной задержки. Зато обе отметки снимаются одним процессом, и согласование таймеров разных узлов не требуется.

Каждому GPU соответствует отдельный MPI-процесс. Используется Open MPI [6] в режиме CUDA-aware, в котором вызовы MPI принимают указатели на память GPU, и транспортный слой UCX [7].

Среды передачи. В среде `auto` указатели на буферы в памяти GPU передаются непосредственно в вызовы MPI, а путь передачи выбирает UCX. После приёма выполняется синхронизация устройства, поскольку транспорт CUDA IPC может сообщить о завершении приёма раньше, чем копирование в память GPU фактически завершится. В среде `host` программа сама копирует данные в оперативную память и передаёт их средствами MPI, а для пар внутри одного узла использует общий буфер в разделяемой памяти узла.

Режимы тестирования. В режиме `one_to_one` пары GPU перебираются последовательно, и в каждый момент времени передача идёт только между одной парой. В режиме `all_to_all` передачи между всеми парами идут одновременно. Начальная отметка у получателя в этом режиме общая для всех источников, а конечная фиксируется отдельно для каждого источника.

3. Настройка коммуникационного стека

3.1 Транспорты и протоколы UCX

В обеих средах Open MPI передаёт сообщения через UCX (`OMPI_MCA_pml=ucx`) с включённой поддержкой CUDA (`OMPI_MCA_opal_cuda_support=1`). Путь передачи определяет UCX: для каждой пары процессов он выбирает транспорт из списка,

заданного переменной `UCX_TLS` (таблица 1). В среде `auto` в список входят транспорты `CUDA`, а переменная `UCX_IB_GPU_DIRECT_RDMA=y` разрешает сетевому адаптеру обращаться к памяти GPU. В среде `host` в MPI передаются только указатели на оперативную память, поэтому транспорты `CUDA` исключены, а `GPUDirect RDMA` запрещена.

Транспорт	Назначение	<code>auto</code>	<code>host</code>
<code>cuda_copy</code>	копирование между памятью GPU и оперативной памятью	+	—
<code>cuda_ipc</code>	доступ к памяти GPU другого процесса узла (GPUDirect P2P)	+	—
<code>rc, ud, dc</code>	передача между узлами по сети InfiniBand	+	+
<code>cma</code>	копирование между процессами узла средствами ядра ОС	+	+
<code>sm</code>	передача через разделяемую память узла	+	+
<code>self</code>	передача внутри процесса	+	+

Таблица 1: Транспорты UCX в средах `auto` и `host`

Если нужный транспорт не указан в списке или недоступен, UCX не сообщает об ошибке, а передаёт данные другим путём, как правило через оперативную память. По результатам это заметно лишь по скорости передачи. Поэтому модуль выставляет переменные UCX до вызова `MPI_Init`, внутри которого инициализируется UCX, и записывает их значения в заголовок каждого файла результатов. Переменная выставляется, только если она не задана в окружении, поэтому скрипт запуска перед каждым запуском сбрасывает значения, унаследованные из окружения пользователя.

Сообщение передаётся одним из двух протоколов. В протоколе `eager` отправитель сразу посылает данные, не дожидаясь готовности получателя, и получатель при необходимости складывает их во временный буфер. Сообщение в памяти GPU при этом копируется через оперативную память (пути `b` и `d` на рис. 1). В протоколе `rendezvous` отправитель сначала посылает только запрос, и данные передаются после того, как получатель подготовит буфер приёма. Это требует дополнительного обмена служебными сообщениями, но позволяет передать данные без промежуточного копирования (пути `a` и `c`). Обычно протокол `eager` выгоден для небольших сообщений, поскольку не требует согласования, а `rendezvous` — для крупных. Однако для сообщений

в памяти GPU eager требует копирования через оперативную память, и его преимущество может теряться (раздел 5.1).

Выбор протокола определяется порогом `UCX_RNDV_THRESH`, который задаётся при запуске: сообщения меньше порога передаются протоколом eager, остальные — протоколом rendezvous. Использовались три значения порога. Значение по умолчанию UCX выбирает автоматически. При значении 0 любое сообщение передаётся протоколом rendezvous. Третье значение выбрано больше наибольшего размера сообщения в измерениях, поэтому все сообщения передаются протоколом eager. Два крайних значения позволяют сравнить оба протокола на всём диапазоне размеров.

3.2 Привязка процессов к ядрам, GPU и сетевым адаптерам

Процессы запускаются командой `srun` с интерфейсом PMIx, по одному на GPU. Флаг `--gres-flags=enforce-binding` требует, чтобы ядра процесса находились на том же сокете, что и его GPU [8], иначе обмен между процессором и GPU пойдёт через межпроцессорное соединение. Ядра закрепляются за процессом параметром `--cpu-bind=cores`.

GPU запрашиваются на узел (`--gpus-per-node`), а не на процесс (`--gpus-per-task`). Во втором случае процесс видит только свой GPU, и транспорт CUDA IPC между процессами узла не работает. Параметр `--gpu-bind` тоже не используется: вариант `closest` выбирает GPU для каждого процесса независимо и при одинаковой близости GPU к процессорам может выдать нескольким процессам один и тот же GPU. Поэтому всем процессам видны все GPU узла (`--gres-flags=allow-task-sharing`), а программа назначает GPU по номеру процесса внутри узла.

По умолчанию UCX выбирает сетевой адаптер, ближайший к ядрам процесса. Однако при передаче без промежуточного копирования данные идут между GPU и адаптером мимо процессора. Поэтому модуль выбирает среди адаптеров InfiniBand тот, путь к которому в дереве устройств PCIe имеет наиболее длинный общий префикс с путём к GPU, то есть находится за общим с ним коммутатором PCIe. Имя адаптера с номером порта (например, `mlx5_0:1`) записывается в `UCX_NET_DEVICES`. Эта переменная должна быть задана до `MPI_Init`, поэтому номер процесса внутри узла берётся не средствами MPI, а из переменных окружения системы запуска (`SLURM_LOCALID`).

4. Экспериментальная среда и методика измерений

Измерения выполнены на вычислительном комплексе ЦКП «Информатика» [9] на узлах двух типов с GPU NVIDIA A100 40 ГБ. Узлы xFusion G5500 V7 содержат по четыре GPU, подключённых по шине PCIe, и по два процессора Intel Xeon Gold 6444Y. Узлы xFusion G8600 V7 содержат по восемь GPU в исполнении SXM4, объединённых коммутаторами NVSwitch, и по два процессора Intel Xeon Platinum 8462Y+. Узлы соединены сетью InfiniBand 100 Гбит/с. Использовались CUDA Toolkit 12.8, Open MPI 5.0.10 и UCX 1.15.0. UCX собран без библиотеки GDRCopy, ускоряющей копирование небольших сообщений из памяти GPU.

В измерениях использовались по два GPU на двух узлах G5500 V7 и четыре GPU на одном узле G8600 V7. Таким образом, рассматриваются упорядоченные пары GPU трёх типов: 12 пар внутри узла G8600 V7 (NVLink), 4 пары внутри узлов G5500 V7 (PCIe) и 8 пар на разных узлах G5500 V7 (PCIe и InfiniBand). Теоретическая пропускная способность NVLink третьего поколения, установленного на A100, составляет 300 ГБ/с в одном направлении, а сети InfiniBand 100 Гбит/с — 12.5 ГБ/с.

Размер сообщения S изменялся от 1 КБ до 16 МБ, и для каждой пары GPU и каждого размера выполнялась серия замеров. Результаты приводятся для режима `one_to_one`, в котором передачи разных пар не конкурируют между собой и задержка определяется путём передачи.

Первые передачи в каждой серии не учитываются: так из результата исключаются возможные разовые издержки на установление соединений транспортного слоя и регистрацию буферов. В замерах внутри серии могут наблюдаться периодические всплески задержки [10], поэтому типичное значение для пары оценивается не средним, а медианой $\tilde{t}_{ij}$, устойчивой к выбросам [11]. Для каждого типа пар и размера сообщения приводится медиана по парам $\tilde{t} = \text{med}_{(i,j)} \tilde{t}_{ij}$, а эффективная скорость передачи оценивается как $B = S/\tilde{t}$.

Для крупных сообщений зависимость задержки от размера сообщения описывается линейной моделью Хокни [12]

$$\tilde{t}(S) = \alpha + S/\beta,$$

где α — постоянная составляющая задержки, а β — асимптотическая скорость передачи. Параметры модели оцениваются методом наименьших квадратов по размерам от 2 до

16 МБ. В отличие от эффективной скорости B , параметр β не включает постоянную составляющую и характеризует сам путь передачи.

5. Результаты

5.1 Влияние порога протокола UCX

Поскольку протокол выбирается по размеру сообщения, в среде `auto` от размера зависит и путь передачи. Порог по умолчанию различается между группами узлов: переход происходит между 8 и 16 КБ на узле G8600 V7 и между 256 и 512 КБ на узлах G5500 V7. Ниже порога задержку ограничивает копирование, а не канал. На узлах G5500 V7 при 128 КБ задержки для пар внутри узла и для пар на разных узлах совпадают (213 мкс), а при переходе через порог задержка падает в 3.3 и в 2.6 раза соответственно (рис. 2).

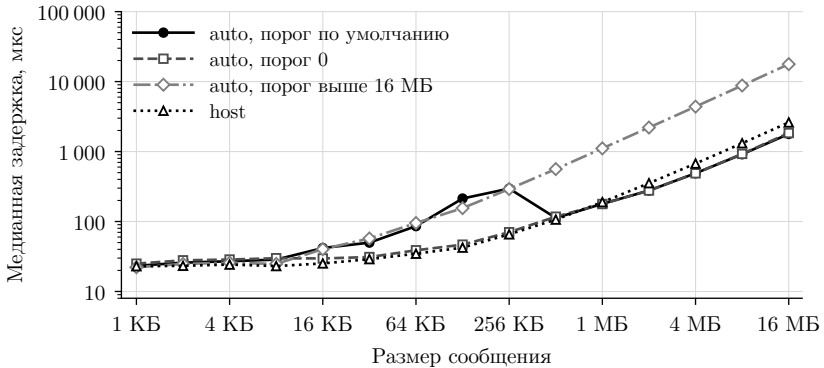


Рис. 2: Задержка для пар GPU на разных узлах G5500 V7 (PCIe + InfiniBand) в среде `auto` при трёх значениях `UCX_RNDV_THRESH` и в среде `host`, режим `one_to_one`

При пороге выше 16 МБ все сообщения передаются протоколом `eager`, и при 16 МБ скорость передачи составляет 0.77–0.92 ГБ/с для всех трёх типов пар, в 7–17 раз медленнее среды `host`. По-видимому, без GDRCopy UCX копирует данные из памяти GPU небольшими порциями.

Чтобы путь передачи не зависел от размера сообщения и группы узлов, основные измерения выполнены с `UCX_RNDV_THRESH=0`, при котором любое сообщение передаётся протоколом `rendezvous`. На узле G8600 V7 это снижает задержку

для сообщений 1 КБ с 20.2 до 12.7 мкс. На узлах G5500 V7 плата за согласование передачи невелика: задержка для сообщений 1 КБ возрастает на 0.4–1.7 мкс. Таким образом, на рассмотренных узлах протокол rendezvous почти не уступает eager даже для небольших сообщений: выигрыш от отсутствия согласования перекрывается затратами на копирование из памяти GPU.

5.2 Задержка и скорость передачи

Далее приводятся результаты при `UCX_RNDV_THRESH=0`. Медианные задержки для сообщений 1 КБ и 16 МБ и параметры модели Хокни приведены в таблице 2. Для сообщений 1 КБ задержка в среде `auto` составляет 12.7 мкс для пар внутри узла G8600 V7, 23.4 мкс для пар внутри узла G5500 V7 и 25.0 мкс для пар на разных узлах G5500 V7. В среде `host` она близка к 23 мкс для всех трёх типов пар.

Пары	Среда	$\tilde{t}$, мкс		α , мкс	β , ГБ/с
		1 КБ	16 МБ		
NVLink	<code>auto</code>	12.7	72.0	13	278
	<code>host</code>	22.9	1258	25	13.3
PCIe	<code>auto</code>	23.4	834	84	21.8
	<code>host</code>	22.7	1256	31	13.4
PCIe + IB	<code>auto</code>	25.0	1840	44	9.15
	<code>host</code>	23.0	2606	30	6.36

Таблица 2: Медианная задержка и параметры модели Хокни для пар внутри узла G8600 V7 (NVLink), внутри узла G5500 V7 (PCIe) и на разных узлах G5500 V7 (PCIe + IB)

Для сообщений от 2 до 16 МБ модель отклоняется от измерений не более чем на 4%. В среде `auto` асимптотическая скорость передачи для пар внутри узла G8600 V7 составляет 278 ГБ/с, то есть 93% теоретической пропускной способности NVLink, а эффективная скорость при 16 МБ — 228 ГБ/с. Для пар внутри узла G5500 V7 асимптотическая скорость составляет 21.8 ГБ/с, для пар на разных узлах G5500 V7 — 9.15 ГБ/с, или 73% пропускной способности InfiniBand. Задержки отдельных пар одного типа различаются мало: при 16 МБ разброс между парами не превышает 2%. Таким образом, в режиме `one_to_one` неоднородность задержек определяется способом соединения GPU, а не отдельными парами.

В среде `host` асимптотическая скорость для пар внутри узла почти одинакова на обеих группах узлов (13.3 и 13.4 ГБ/с), то есть

NVLink в этой среде не используется. Передача внутри узла здесь состоит из двух последовательных копирований: из памяти GPU отправителя в общий буфер и из него в память GPU получателя. Если скорости копирования в обе стороны одинаковы и равны β_c , то $1/\beta = 2/\beta_c$, откуда $\beta_c \approx 26.7$ ГБ/с. Для пар на разных узлах к копированиям добавляется передача по сети со скоростью β_n : $1/\beta = 2/\beta_c + 1/\beta_n$, откуда $\beta_n \approx 12.1$ ГБ/с, то есть 97% пропускной способности InfiniBand.

Следовательно, в среде `host` сеть InfiniBand используется почти полностью, а при передаче с GPUDirect RDMA в среде `auto` — лишь на 73%. По-видимому, передачу без промежуточного копирования между узлами G5500 V7 ограничивает путь по шине PCIe между сетевым адаптером и памятью GPU. Среда `host` проигрывает потому, что копирования и передача по сети выполняются в ней последовательно. Если бы они перекрывались, как при конвейерной передаче, скорость для крупных сообщений могла бы приблизиться к 12 ГБ/с и превзойти скорость GPUDirect RDMA. Проверка этого предположения выходит за рамки работы.

5.3 Выигрыш от передачи без промежуточного копирования

На рис. 3 показано отношение медианной задержки в среде `host` к задержке в среде `auto`. Внутри узла G8600 V7 среда `auto` выигрывает при любом размере сообщения: не менее чем в 1.4 раза для сообщений до 16 КБ и в 17.5 раза при 16 МБ.

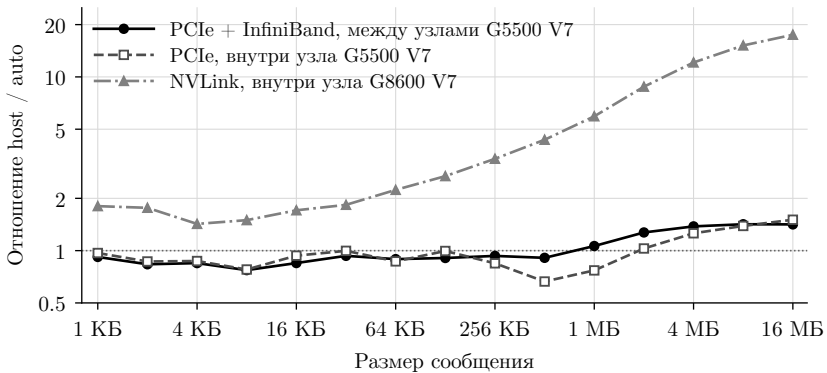


Рис. 3: Отношение медианной задержки в среде `host` к задержке в среде `auto` при `UCX_RNDV_THRESH=0`, режим `one_to_one`. Значения больше 1 соответствуют выигрышу среды `auto`

На узлах G5500 V7 картина иная. Для сообщений до 512 КБ между узлами и до 1 МБ внутри узла среда `auto` не быстрее среды `host`, а при 512 КБ внутри узла медленнее её в 1.5 раза. Выигрыш появляется начиная с 1 МБ между узлами и с 2 МБ внутри узла и при 16 МБ составляет 1.4 и 1.5 раза соответственно.

Для пар внутри узла G5500 V7 это объясняется большой постоянной составляющей задержки в среде `auto`: 84 мкс против 31 мкс в среде `host` (таблица 2). Приравнивая задержки обеих сред по модели Хокни, получаем размер сообщения, при котором они совпадают:

$$S^* = \frac{\alpha_{\text{auto}} - \alpha_{\text{host}}}{1/\beta_{\text{host}} - 1/\beta_{\text{auto}}} \approx 1.8 \text{ МБ},$$

что согласуется с измерениями. При этом для сообщений до 128 КБ задержка для этих пар в среде `auto` не превышает 41 мкс, то есть постоянная составляющая растёт с размером сообщения. Вероятно, это связано со сменой способа передачи внутри UCX, однако установить это по имеющимся данным нельзя.

Таким образом, передача без промежуточного копирования выгодна не всегда. На узлах, где GPU соединены шиной PCIe, для сообщений до 1 МБ передача через оперативную память оказывается не медленнее, а в ряде случаев быстрее.

Заключение

Разработан модуль нагрузочного тестирования, измеряющий задержки передачи данных между всеми упорядоченными парами GPU кластера в двух средах передачи. Путь передачи данных между GPU определяется не только аппаратурой, но и настройками Open MPI, UCX и Slurm, причём при неподходящих настройках UCX не сообщает об ошибке, а передаёт данные через оперативную память. По опыту работы можно сформулировать следующие рекомендации.

- Параметры UCX следует задавать явно, сбрасывать значения, унаследованные из окружения, и сохранять фактические значения вместе с результатами.
- Сетевой адаптер следует выбирать по близости к GPU в топологии PCIe, а не к процессу.
- Ядра процессора следует выделять на сокете GPU (`--gres-flags=enforce-binding`). GPU следует запрашивать

на узел и назначать процессам детерминированно, не полагаясь на `--gpu-bind=closest`.

- Порог протокола rendezvous различается между группами узлов и меняет путь передачи в зависимости от размера сообщения, поэтому его значение должно фиксироваться и указываться вместе с результатами. На рассмотренных узлах порог 0 увеличивает задержку для небольших сообщений не более чем на 2 мкс и заметно снижает её для сообщений среднего размера.

Измерения показали, что выигрыш от передачи без промежуточного копирования зависит от способа соединения GPU. Внутри узла с NVLink он составляет от 1.4 до 17.5 раза во всём диапазоне размеров сообщений. На узлах с PCIe он появляется только для сообщений от 1–2 МБ и не превышает 1.5 раза. Между узлами GPUDirect RDMA использует пропускную способность InfiniBand на 73%, тогда как в среде `host` сама передача по сети идёт со скоростью 97% пропускной способности.

В дальнейшем планируется исследовать режим `all_to_all`, выделить группы пар GPU со сходными задержками аналогично [1] на большем числе узлов, а также оценить конвейерную передачу через оперативную память и влияние GDRCoPy на передачу небольших сообщений.

Литература

1. Сальников А. Н., Бегаев А. А., Майсурадзе А. И. *Кластеризация задержек между узлами суперкомпьютера и визуализация полученных результатов* // Современные проблемы математического моделирования, обработки изображений и параллельных вычислений 2017: труды Международной научной конференции (пос. Дивноморское, 4–11 сентября 2017 г.). Ростов-на-Дону: ДГТУ-Принт, 2017. Т. 1. С. 206–215.
2. Бегаев А. А., Сальников А. Н. *Метод измерения задержек при передаче данных между графическими процессорами, находящимися на разных узлах вычислительного кластера* // Вестник Московского университета. Серия 15: Вычислительная математика и кибернетика. 2020. Вып. 1. С. 3–13.

3. *Clustbench / network-tests2, вемка gpu_tests*. https://github.com/clustbench/network-tests2/tree/gpu_tests (дата обращения: 28.09.2026).
4. NVIDIA. *CUDA C++ Programming Guide: Peer Device Memory Access*. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/> (дата обращения: 20.09.2026).
5. NVIDIA. *GPUDirect RDMA*. <https://docs.nvidia.com/cuda/gpudirect-rdma/> (дата обращения: 20.09.2026).
6. *Open MPI 5.0.x Documentation: CUDA*. <https://docs.openmpi.org/en/v5.0.x/tuning-apps/networking/cuda.html> (дата обращения: 20.09.2026).
7. *Unified Communication X*. <https://openucx.org/> (дата обращения: 20.09.2026).
8. *Slurm Workload Manager: Generic Resource (GRES) Scheduling*. <https://slurm.schedmd.com/gres.html> (дата обращения: 20.09.2026).
9. Центр коллективного пользования «Информатика» ФИЦ ИУ РАН: *Вычислительная инфраструктура*. <https://docs.hpc.frcsc.ru/hardware/> (дата обращения: 20.09.2026).
10. Волчанинов А. П., Сальников А. Н. *Исследование структуры серии измерений задержки при нагрузочном тестировании коммутационной среды вычислительного кластера // Суперкомпьютерные дни в России: труды международной конференции (Москва, 25–26 сентября 2023 г.)*. М.: МАКС Пресс, 2023. С. 42–52.
11. Hoeffler T., Belli R. *Scientific Benchmarking of Parallel Computing Systems: Twelve Ways to Tell the Masses when Reporting Performance Results* // *Proceedings of SC'15*. 2015. Art. 73. P. 1–12. DOI: 10.1145/2807591.2807644.
12. Hockney R. W. *The communication challenge for MPP: Intel Paragon and Meiko CS-2* // *Parallel Computing*. 1994. Vol. 20, no. 3. P. 389–398.